

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
AI and application Development and jira

Practical – 7- F,G& H

Roll No.:T050	Name: Aashu Yadav
Class:TYIT	Batch:1
Date of Assignment:08-8-2026	Date/Time of Submission:08-8-2026

Aim:-6. Implement Decision Tree regression on a regression dataset.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load dataset
data = load_iris()
# Input and output
X = data.data
y = data.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create model
model = DecisionTreeClassifier()
# Train model
```

```
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train)
* 100)
print("Testing Accuracy:", model.score(X_test, y_test) *
100)
# Prediction
new_data = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_data)
# Display prediction
print("Predicted class:", prediction[0])
print("Predicted flower:", data.target_names[prediction[0]])
print("Aashu 50")
```

Output:-

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted class: 0
Predicted flower: setosa
Aashu 50
```

Aim:-7. Implement k-Nearest Neighbors (kNN) classifier.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.3,
    random_state=1
)
# Create kNN model
model = KNeighborsClassifier(n_neighbors=3)
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train,
```

```
y_train)*100)
print("Testing Accuracy:", model.score(X_test,
y_test)*100)
# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
print("Aashu 50")
```

Aim:-8. Implement multiclass classification using Logistic Regression or Decision Tree.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=1
)
# Create Decision Tree model
model = DecisionTreeClassifier()
# Train model
```

Output:-

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Class: 0
Aashu 50
```

```
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train,
y_train)*100)
print("Testing Accuracy:", model.score(X_test,
y_test)*100)
# Predict one sample
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
print("Aashu 50")
```

Output:-

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
Aashu 50
```

Aim:- Perform Logistic Regression, Decision Tree, and kNN on the same dataset. Compare the Results.

Code:-

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
```

```
# Logistic Regression
model1 = LogisticRegression(max_iter=10000)
model1.fit(X_train, y_train)
print("Logistic Regression")
print("Training Accuracy:", model1.score(X_train, y_train)
* 100)
print("Testing Accuracy:", model1.score(X_test, y_test) *
100)
# Decision Tree
model2 = DecisionTreeClassifier()
model2.fit(X_train, y_train)
print("\nDecision Tree")
print("Training Accuracy:", model2.score(X_train, y_train)
* 100)
print("Testing Accuracy:", model2.score(X_test, y_test) *
100)
```

Aashu Yadav

```
# kNN
model3 = KNeighborsClassifier(n_neighbors=3)
model3.fit(X_train, y_train)
print("\nkNN")
print("Training Accuracy:", model3.score(X_train, y_train))
```

```
* 100)
print("Testing Accuracy:", model3.score(X_test, y_test) *
100)
print("Aashu 50")
```

varies on the same dataset.

Output:-

```
Logistic Regression
Training Accuracy: 96.23115577889448
Testing Accuracy: 94.73684210526315

Decision Tree
Training Accuracy: 100.0
Testing Accuracy: 95.32163742690058

kNN
Training Accuracy: 94.72361809045226
Testing Accuracy: 92.39766081871345
Aashu 50
```

Algorithm	Training Accuracy	Testing Accuracy
Logistic Regression	~96%	~95%
Decision	100%	~95%
kNN	~94%	~93%

Justification

The algorithms produced different accuracies because they use different classification methods. Logistic Regression uses probability, Decision Tree uses decision rules, and kNN uses nearest neighbors. Therefore, their performance

Conclusion

Logistic Regression and Decision Tree performed better than kNN. Decision Tree showed 100% training accuracy, while Logistic Regression gave a good overall performance